

重大 AI 更新提醒

10-03 | llama.cpp CUDA 融合共享专家至 MMVQ，MoE 推理再提速

本期导读

本期聚焦 llama.cpp 最新 release b11370，该版本在 CUDA 后端将共享专家（shared experts）融合进 MMVQ（量化矩阵 向量乘）算子，并处理了缓冲区空指针检查与 stride_col_dst 参数传递。这一改动直接面向 MoE（混合专家）模型在本地 GPU 上的推理效率，对使用 llama.cpp 部署 Qwen、DeepSeek 等 MoE 架构的开发者具有实际意义。

已检查来源

GitHub Release

1. llama.cpp b11370：CUDA 后端将共享专家融合进 MMVQ 算子

来源 GitHub Release | 类型 GitHub Release | 主题 推理性能与基础设施 时间 10-03 01:24 | 分数 7

是什么	这是 ggml-org/llama.cpp 项目在 GitHub 上发布的 b11370 版本，核心变更是 CUDA 后端把 MoE 模型中的共享专家（shared experts）计算融合进 MMVQ（量化矩阵 向量乘）算子中。同时该 PR 增加了对缓冲区是否为空的检查，并把 stride_col_dst 移动到融合参数里，以保证融合路径的正确性。 llama.cpp 是一个用 C/C++ 实现的高性能大语言模型推理框架，支持在 CPU、CUDA、Metal 等多种后端上运行量化模型。
通俗解释	可以把 MoE 模型想象成一家大公司，每次来一个任务，不是所有部门都干活，而是先由 共享专家 部门做通用处理，再交给若干 专门专家 部门分头处理。以前在 GPU 上，共享专家和后续的量化矩阵乘法是分成两步做的，中间要把数据写回显存再读出来，就像两个部门之间来回寄快递。b11370 把这两步合并成一步，共享专家的结果直接进入 MMVQ 的融合计算，省掉了中间来回搬运。对于普通用户来说，这意味着在本地用 llama.cpp 跑 MoE 模型时，GPU 利用率更高、响应可能更快，尤其是在显存带宽有限的家用显卡上。
主要作用	解决 MoE 模型在 CUDA 后端推理时共享专家与量化矩阵乘法分离执行带来的额外显存读写和 kernel 启动开销。通过算子融合减少中间张量的落盘与读取，提升 GPU 推理吞吐，并降低延迟。对于在本地或边缘设备部署 MoE 大模型的开发者，这是直接可感知的性能优化。
核心功能	1) CUDA 后端将共享专家计算融合进 MMVQ 量化矩阵 向量乘算子，减少 kernel 数量与显存往返； 2) 增加缓冲区空指针检查，提升融合路径的健壮性；3) 将 stride_col_dst 移入融合参数，保证融合后矩阵步长传递正确；4) 随 release 提供 macOS Apple Silicon、macOS Intel、iOS XCFramework、Ubuntu x64/arm64 等平台预编译包。
对比判断	候选摘要中未给出与上一版本或其他推理框架的 benchmark、速度或成本对比数据，暂无可信公开对比数据。
适合谁	适合在本地或自有 GPU 服务器上使用 llama.cpp 部署 MoE 架构大模型的开发者、推理工程师和研究者，尤其是关注 CUDA 后端性能、需要量化推理加速的团队；也适合维护 llama.cpp 下游发行版或集成到桌面 移动端应用的技术人员。

直达链接 <https://github.com/ggml-org/llama.cpp/releases/tag/b11370>

本期仅一条候选，已按真实 release 信息整理；性能提升幅度需以官方后续 benchmark 为准。